nn.Module 기초: 층 하나에서 모델까지

코드 리뷰 b01 ~ b10 · 2권 6장 6.1 ~ 6.4

오늘 볼 코드 10개

묶음	파일	핵심 질문	슬라이드
1. 층 하나	b01 손으로 만든 층 · b02 nn.Linear · b03 선형층 두 개	층은 무슨 계산을 하나? 선형층을 쌓으면 더 똑똑해질까?	3~7
2. 활성화	b04 활성화 함수 · b05 softmax · b06 XOR	활성화 함수가 없으면 무엇을 못 하나?	8~12
3. 모델 만들기	b07 nn.Module · b08 Sequential · b09 파라미터 · b10 ModuleList	모델은 어떻게 만들고, 가중치는 몇 개이고 어디에 저장되나?	13~17

코드 하나를 볼 때마다

① 실행 전에 결과를 먼저 예측 → ② 실행해서 비교 → ③ 왜 그런지 한 줄 정리 → ④ 숫자나 줄 하나 바꿔 보기

준비

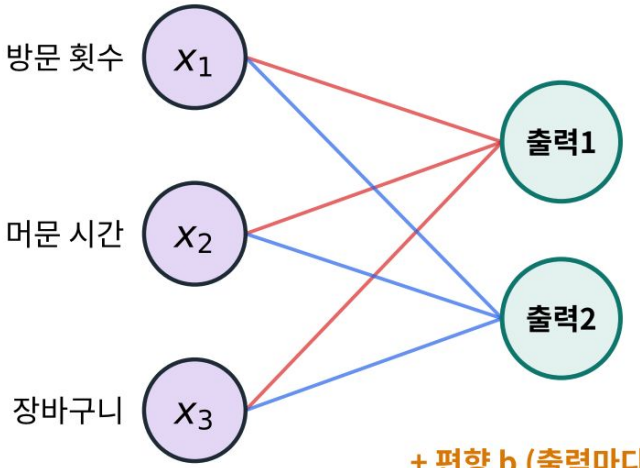
torch만 있으면 됨 (CPU로 충분). 파일마다 따로 실행하는 짧은 코드. 마지막에 코드 고치기 문제 6개와 정답.

1 층 하나: 입력 @ 가중치 + 편향

b01 손으로 만든 층 · b02 nn.Linear · b03 선형층 두 개

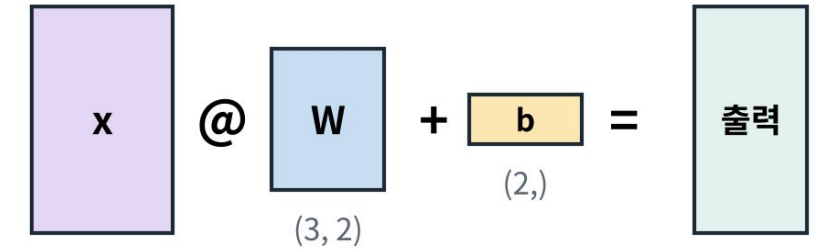
개념: 층 하나 = 가중합을 한꺼번에

선마다 가중치 w 하나 → 3 × 2 = 6개



+ 편향 b (출력마다 1개)

안쪽 숫자 3끼리 맞아야 곱할 수 있음



출력 = x @ W + b

@ = 행렬 곱. 손님 4명을 한 번에 계산 (배치). 가중치 개수 = 입력 수 × 출력 수

b01 손으로 만든 층

```
x = torch.tensor([[0.2, 0.5, 0.1],   #손님 4명 × 특성 3개
                  [0.9, 0.1, 0.4],
                  [0.4, 0.8, 0.7],
                  [0.6, 0.3, 0.2]])
torch.manual_seed(0)
가중치 = torch.randn(3, 2)     # 입력 3개 → 출력 2개
편향 = torch.zeros(2)          # 출력마다 하나씩

출력 = x @ 가중치 + 편향         # (4, 3) @ (3, 2) + (2,) → (4, 2)

# 첫 손님의 첫 번째 출력을 풀어 쓰면
print((x[0] * 가중치[:, 0]).sum() + 편향[0])
```

```
출력
tensor([[-0.8896,  0.0857],
        [ 0.7352, -0.7667],
        [-1.8858, -0.6416],
        [ 0.0541, -0.2852]])
모양: torch.Size([4, 2])
tensor(-0.8896)   ← 같은 값
```

코드	의미
x[0]	첫 손님 (행)
가중치[:, 0]	0번 출력으로 가는 w 3개 (열)
+ 편향 (2,)	모든 행에 같이 더해짐 (브로드캐스팅)

첫 칸 풀어 쓰기

x[0] = [0.2, 0.5, 0.1], 가중치 0번 열 = [1.541, −2.179, −1.085]
0.2×1.541 + 0.5×(−2.179) + 0.1×(−1.085) + 0 = −0.890

b02 nn.Linear: PyTorch가 만들어 주는 층

```
torch.manual_seed(0)
층 = nn.Linear(3, 2)              #입력 3개 → 출력 2개
print(층)
print("weight:", 층.weight.shape, "bias:", 층.bias.shape)

x = torch.rand(4, 3)
print(층(x))
직접 = x @ 층.weight.T + 층.bias  # 손으로 같은 계산
print("같은가:", torch.allclose(층(x), 직접))
print("학습 대상인가:", 층.weight.requires_grad)
```

```
Linear(in_features=3, out_features=2, bias=True)
weight: torch.Size([2, 3]) bias: torch.Size([2])
tensor([[ 0.0166,  0.1776], ...  grad_fn=<AddmmBackward0>)
같은가: True
학습 대상인가: True
```

	b01 손으로	b02 nn.Linear
가중치 모양	(3, 2) 입력 × 출력	**(2, 3) 출력 × 입력**
계산	x @ W + b	x @ W.T + b
처음 값	직접 정함 (randn, zeros)	자동으로 작은 랜덤 값
학습 대상	따로 설정해야 함	자동 (True)

왜 (출력, 입력)일까?

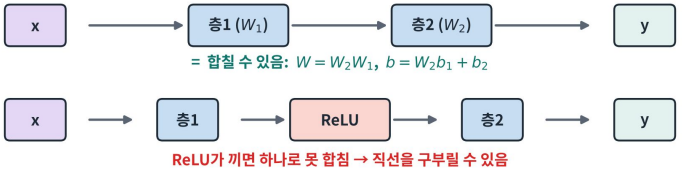
한 행 = 출력 뉴런 하나의 w 묶음. 0번 행이 '출력1로 가는 w 3개'라서 뉴런 단위로 보기 편함. 그래서 계산할 때 .T(전치)로 뒤집음.

grad_fn = 역전파에 쓸 계산 기록이 붙어 있다는 표시 (5장 자동 미분)

b03 선형층 두 개 = 선형층 한 개

```
층1 = nn.Linear(3, 5)
층2 = nn.Linear(5, 2)
두_층 = 층2(층1(x))

# 두 층을 합친 가중치 하나, 편향 하나
W = 층2.weight @ 층1.weight          # (2, 3)
b = 층2.weight @ 층1.bias + 층2.bias  # (2,)
한_층 = x @ W.T + b
print("같은가:", torch.allclose(두_층, 한_층, atol=1e-6))
# → 같은가: True
```



왜 같을까?

층2(층1(x)) = W_2 (W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)
곱하기와 더하기만 있으면 아무리 쌓아도 결국 '하나의 곱하기 + 더하기'로 정리됨 → 직선(평면) 경계만 가능

그래서 다음 묶음

층 사이에 직선이 아닌 함수(활성화 함수)를 끼워야 깊게 쌓는 의미가 생김 → b04 ~ b06

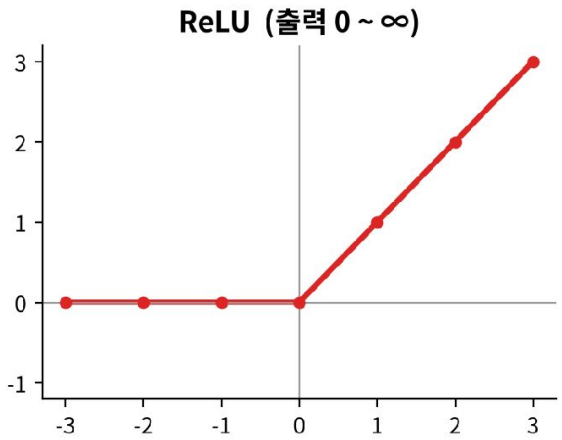
해 보기: 층2(torch.relu(층1(x)))로 바꾸면 같은가: False

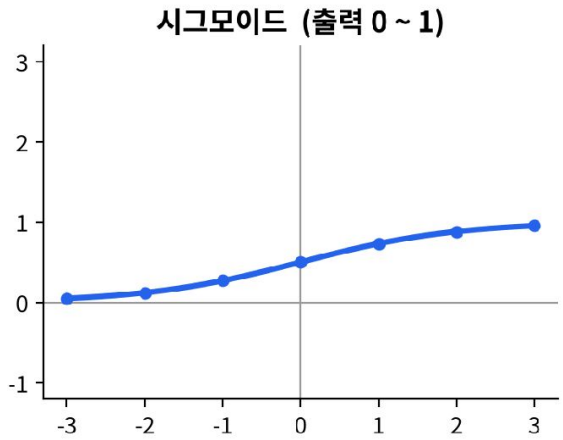
2

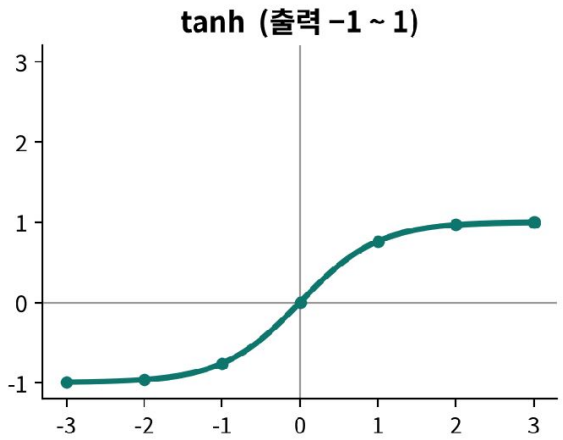
활성화 함수: 직선을 구부리는 장치

b04 ReLU · 시그모이드 · tanh · b05 softmax · b06 XOR

b04 활성화 함수 세 가지





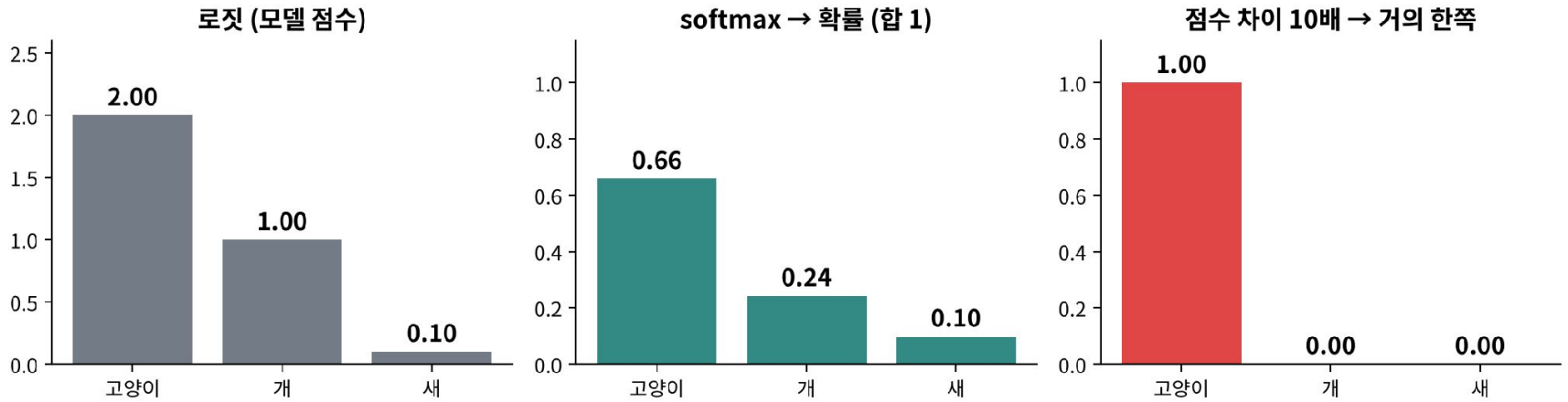


입력	−3	−1	0	1	3	쓰는 곳
ReLU	0	0	0	1	3	은닉층 기본 (계산 빠름, CNN·MLP)
시그모이드	0.05	0.27	0.5	0.73	0.95	이진 분류 확률, 게이트 (LSTM)
tanh	−0.99	−0.76	0	0.76	0.99	RNN 은닉 상태

```
torch.relu(x)          # 함수로 쓰기
활성화 = nn.ReLU()      # 층처럼 쓰기
활성화(x)              # 결과 같음
```

nn.ReLU()는 Sequential에 층처럼 넣을 때 씀 (b06, b08)

b05 softmax: 점수를 확률로



$$\text{softmax}(z_i) = e^{z_i} / \Sigma\, e^{z\square}$$

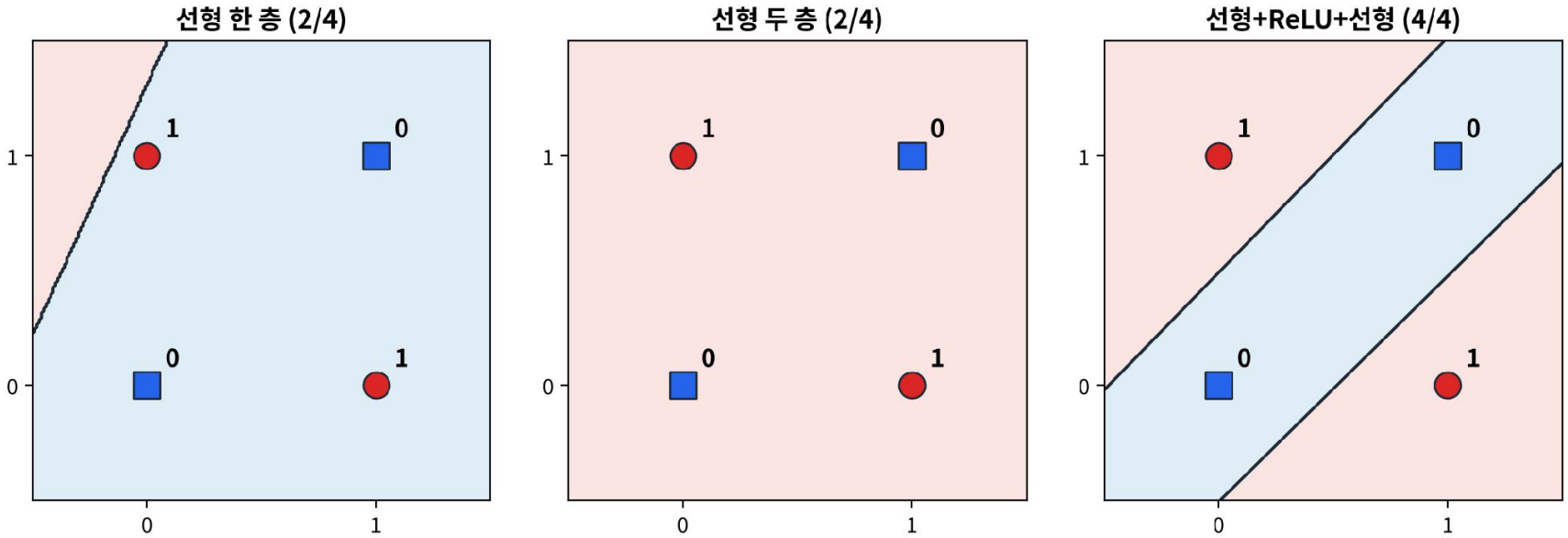
e^z로 모두 양수 → 합으로 나눠 합 1

dim=1

행(사진)마다 클래스 3개를 합 1로. dim=0이면 사진끼리 합 1이 돼서 틀림

```
로짓 = torch.tensor([[2.0, 1.0, 0.1],   #  사진 2장 × 클래스 3개
                     [0.5, 0.5, 3.0]])
확률 = torch.softmax(로짓, dim=1)       #  클래스 방향으로 합 1
# [[0.659, 0.242, 0.099], [0.071, 0.071, 0.859]]
확률.argmax(dim=1),  로짓.argmax(dim=1)  #  둘 다 [0, 2]
torch.softmax(torch.tensor([1000., 0.]), dim=0)  # [1., 0.]   에러 없음
```

b06 XOR: 직선 하나로는 못 나누는 문제



모델	확률 (00, 01, 10, 11)	맞힘	이유
선형 한 층	[0.5, 0.5, 0.5, 0.5]	2/4	직선 경계 하나 → 포기하고 전부 0.5
선형 두 층	[0.5, 0.5, 0.5, 0.5]	2/4	b03: 합치면 한 층과 같음
선형+ReLU+선형	[0.0, 1.0, 1.0, 0.0]	4/4	ReLU로 경계를 꺾어 띠 모양으로 나눔

b06 코드 읽기: 학습 함수와 후보 모델

```
def 학습하기(모델, 단계=2000):
    손실함수 = nn.BCEWithLogitsLoss()      # 0/1  분류용 손실
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=0.05)
    for _ in range(단계):
        옵티마이저.zero_grad()
        손실 = 손실함수(모델(X), y)
        손실.backward()
        옵티마이저.step()
    return 모델

후보 = {
  "선형 한 층": lambda: nn.Linear(2, 1),
  "선형 두 층": lambda: nn.Sequential(nn.Linear(2, 4), nn.Linear(4, 1)),
  "선형+ReLU+선형": lambda: nn.Sequential(nn.Linear(2, 4), nn.ReLU(),
                                          nn.Linear(4, 1)),
}
for 이름, 만들기 in 후보.items():
    torch.manual_seed(1)
    모델 = 학습하기(만들기())
    with torch.no_grad():                  #  결과만 볼 때는 기록 끄기
        확률 = torch.sigmoid(모델(X)).squeeze(1)
```

코드	의미
lambda: …	모델을 바로 만들지 않고 '만드는 방법'을 저장. 시드를 정한 뒤 만들려고
BCEWithLogits	시그모이드 + 0/1 손실을 한 번에. 그래서 결과 볼 때 sigmoid를 따로 씌움
manual_seed(1)	세 모델 모두 같은 조건(같은 시작 랜덤값)에서 비교

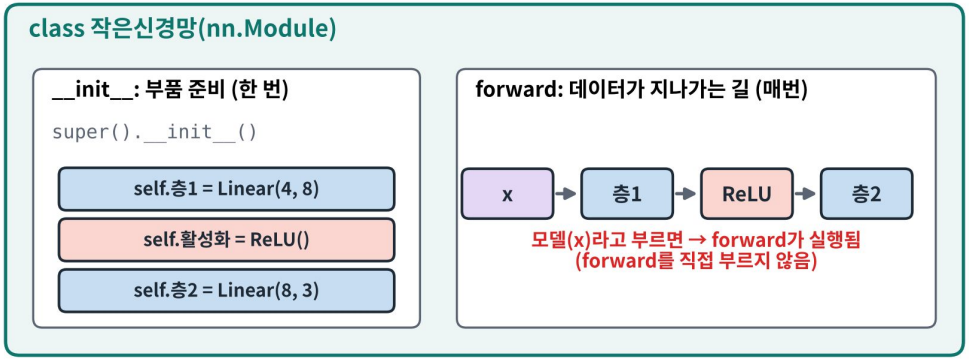
시드를 바꿔 보면

시드 2: ReLU 모델도 [0.5, 0.5, 0.5, 0.5] 2/4로 실패
은닉 4 → 8: 시드 2도 4/4 성공
은닉이 적으면 시작값에 따라 실패할 수 있음
(시드 0~19 중 은닉 4는 12번, 8은 18번 성공)

3 nn.Module로 모델 만들기

b07 클래스 · b08 Sequential · b09 파라미터 · b10 ModuleList

b07 nn.Module: 모든 모델의 기본 틀



규칙	안 지키면
nn.Module을 물려받기	parameters(), .to() 같은 기능을 못 씀
super().__init__() 먼저	층을 self에 넣을 때 에러
층은 self.이름 = …로 저장	지역 변수면 모델이 층을 모름
계산은 forward에, 호출은 모델(x)	훅·모드 처리가 빠짐

```
class 작은신경망(nn.Module):
    def __init__(self, 입력수, 은닉수, 출력수):
        super().__init__()      #  준비 작업
먼저
        self.층1 = nn.Linear(입력수, 은닉수)
        self.활성화 = nn.ReLU()
        self.층2 = nn.Linear(은닉수, 출력수)

    def forward(self, x):
        x = self.층1(x)
        x = self.활성화(x)
        return self.층2(x)

모델 = 작은신경망(4, 8, 3)
출력 = 모델(torch.rand(5, 4))    # (5, 3)
```

출력: torch.Size([5, 3]) · forward와 같은가: True · 파라미터 4×8+8+8×3+3 = 67개

b08 nn.Sequential: 순서대로 쌓기만 할 때

```
모델 = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3),
)
print(모델[1], len(모델))    # ReLU() 3  ← 리스트처럼 꺼내기
print(모델(torch.rand(5, 4)).shape)    # (5, 3)
```

```
사진_모델 = nn.Sequential(
    nn.Flatten(),           # 묶음, 1, 28, 28) → (묶음, 784)
    nn.Linear(28 * 28, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
)
사진들 = torch.rand(32, 1, 28, 28)
# Flatten 결과 (32, 784) → 최종 (32, 10)
```

28 × 28 = 784. Linear는 한 줄(1차원) 입력만 받아서 먼저 펴 줌



b07과 같은 모델

b07 클래스 = 이 Sequential (4 → 8 → ReLU → 3). 층 이름이 0, 1, 2 번호로 붙음

b09 파라미터 들여다보기

```
모델 = nn.Sequential(nn.Flatten(), nn.Linear(784, 256),
                     nn.ReLU(), nn.Linear(256, 10))
for 이름, p in 모델.named_parameters():
    print(이름, tuple(p.shape), p.numel())

for p in 모델[1].parameters():       # 첫 Linear (0번은 Flatten)
    p.requires_grad_(False)          #얼리기: 학습 안 함
```

이름	모양	개수	계산
1.weight	(256, 784)	200,704	784 × 256
1.bias	(256,)	256	출력마다 1개
3.weight	(10, 256)	2,560	256 × 10
3.bias	(10,)	10	
전체		**203,530**	≈ 0.78 MB

이름이 1, 3인 이유

Sequential 번호: 0 Flatten, 1 Linear, 2 ReLU, 3 Linear. Flatten과 ReLU는 파라미터가 없어서 안 나옴

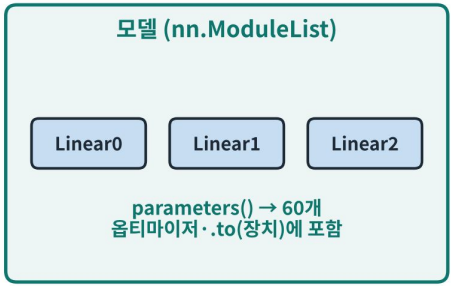
얼리기 (requires_grad=False)

첫 Linear를 얼리면 학습 대상 2,570개만 남음. 이미 학습된 모델의 앞부분은 그대로 두고 뒷부분만 새로 학습할 때 씀 (전이 학습)

가중치 대부분(99%)이 첫 층에 있음 → 입력이 클수록 첫 층이 무거움. CNN이 이미지에서 파라미터를 줄이는 이유와 연결

b10 그냥 list vs nn.ModuleList






```
리스트모델: 파라미터 0개, 출력 (2, 4)
모듈리스트모델: 파라미터 60개, 출력 (2, 4)
```

```
# 리스트모델
self.층들 = [nn.Linear(4, 4) for _ in range(3)]

# 모듈리스트모델
self.층들 = nn.ModuleList([nn.Linear(4, 4) for _ in range(3)])

# forward는 둘 다 같음
for 층 in self.층들:
    x = torch.relu(층(x))
```

무엇이 문제?

계산(출력)은 되는데 모델이 층을 모름
· 옵티마이저: optimizer got an empty parameter list 에러
· .to(장치), 저장(state_dict)에서도 빠짐
· print(모델)에 층이 안 보임

60개 계산

Linear(4, 4) 하나 = 4×4 + 4 = 20 → 3개 = 60

층을 반복문으로 만들 때는 꼭 nn.ModuleList

정리

언제 무엇을 쓸까: 클래스 · Sequential · ModuleList

	nn.Module 클래스 (b07)	nn.Sequential (b08)	nn.ModuleList (b10)
만드는 법	__init__에 층, forward에 계산	층을 순서대로 나열	층을 리스트로 담기
forward	직접 작성 (자유)	자동: 위에서부터 차례로	없음: 클래스 안에서 for로 돌림
잘 맞는 경우	갈래가 나뉘거나 중간값을 여러 번 쓸 때 (RNN, 잔차 연결)	한 줄로 쭉 이어지는 모델 (MLP, 작은 CNN)	같은 층을 N개 반복 (층 수를 인자로 받을 때)
예	작은신경망(4, 8, 3)	Sequential(Linear, ReLU, Linear)	트랜스포머 블록 N개

공통: 모두 nn.Module → parameters(), named_parameters(), .to(장치), train()/eval()을 똑같이 쓸 수 있음

코드 고치기 문제

문제 1 (b01)

가중치를 torch.randn(3, 4)로 바꾸면 출력 모양은? 편향은 어떻게 바꿔야 할까?

문제 2 (b02)

nn.Linear(3, 5)로 바꾸면 weight 모양과 파라미터 수는?

문제 3 (b03)

두_층 = 층2(torch.relu(층1(x)))로 바꾸면 '같은가'는?

문제 4 (b06)

torch.manual_seed(1)을 2로 바꾸면 ReLU 모델 결과는? 어떻게 고칠 수 있을까?

문제 5 (b07)

은닉수를 8에서 16으로 바꾸면 파라미터 수는?

문제 6 (b09, b10)

(1) 첫 층 대신 마지막 Linear(모델[3])를 얼리면 학습 대상은?
(2) 리스트모델로 Adam 옵티마이저를 만들면?

코드 고치기 정답

문제 1

출력 (4, 4). 편향도 torch.zeros(4)로 출력 개수에 맞춤 (안 바꾸면 (2,)과 (4, 4)를 더할 수 없어 에러)

문제 2

weight (5, 3), bias (5,) → 15 + 5 = 20개

문제 3

같은가: False. ReLU가 끼면 두 층을 하나의 W, b로 합칠 수 없음

문제 4

[0.5, 0.5, 0.5, 0.5] 2/4로 실패. 은닉을 8로 늘리면 (Linear(2, 8), Linear(8, 1)) 4/4. tanh로 바꿔도 성공

문제 5

4×16 + 16 + 16×3 + 3 = 131개 (8일 때 67개)

문제 6

(1) 784×256 + 256 = 200,960개
(2) ValueError: optimizer got an empty parameter list

정리

01	**층 하나**	출력 = x @ W + b. nn.Linear는 weight를 (출력, 입력)으로 저장
02	**선형만 쌓으면**	아무리 쌓아도 층 하나와 같음 (W_2 W_1 로 합쳐짐)
03	**활성화 함수**	ReLU 같은 곡선을 끼워야 XOR처럼 꺾인 경계를 배움
04	**softmax**	점수를 합 1인 확률로. 고른 클래스(argmax)는 그대로
05	**nn.Module**	__init__에 층(self.이름), forward에 계산, 호출은 모델(x)
06	**파라미터**	named_parameters로 확인. 반복으로 만든 층은 ModuleList로 등록